

Assignment-8 PL/SQL

- I:

1. Write a PL/SQL block of code to update salary of employee '7788' to 35000 if the salary is less than 35000.

```
SQL> DECLARE
2     salary emp.sal%TYPE;
3 BEGIN
4     SELECT sal INTO salary
5     FROM emp
6     WHERE empno = 7788;
7     IF salary < 35000 THEN
8         UPDATE emp
9         SET sal= 35000
10        WHERE empno = 7788;
11        DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');
```

PL/SQL procedure successfully completed.

2. Write a PL/SQL block of code to insert all details of employee '7698' to a new table temp_emp, which has same structure as emp table.

```
SQL> CREATE TABLE TEMP_EMP AS
2 SELECT * FROM EMP WHERE 1 = 2;

Table created.

SQL> DECLARE
2     newrec EMP%ROWTYPE;
3 BEGIN
4     SELECT * INTO newrec from EMP WHERE EMPNO = 7698;
5     INSERT INTO TEMP_EMP VALUES(newrec.EMPNO, newrec.ENAME, newrec.JOB, newrec.MGR, newrec.HIREDATE, newrec.SAL, newrec.COMM, newrec.DEPTNO);
6     COMMIT;
7 END;
8 /

PL/SQL procedure successfully completed.

SQL> SELECT * FROM TEMP_EMP;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30

3. Write a PL/SQL block of code to display ones name like "Hello <Name>", Whatever entered at run time.

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
2     Name VARCHAR2(20);
3 BEGIN
4     Name := '&Name';
5     DBMS_OUTPUT.PUT_LINE('Hello ' || Name);
6 END;
7 /

Enter value for name: CSE
old 4:   Name := '&Name';
new 4:   Name := 'CSE';
Hello CSE

PL/SQL procedure successfully completed.
```

4. Write a PL/SQL block of code to print first 50 whole no.

```

SQL> BEGIN
2   FOR counter IN 1 .. 50 LOOP
3       DBMS_OUTPUT.PUT_LINE(counter);
4   END LOOP;
5 END;
6 /

```

PL/SQL procedure successfully completed.

5. Write a PL/SQL block of code to update the commission of employee number 7369 to Rs. 3000 if it NULL; else raise his commission by 25%.

```

SQL> DECLARE
2   c EMP.COMM%TYPE;
3 BEGIN
4   SELECT COMM INTO c FROM EMP
5   WHERE EMPNO = 7369;
6   IF c IS NULL THEN
7       UPDATE EMP SET COMM = 3000
8       WHERE EMPNO = 7369;
9   ELSE
10      UPDATE EMP SET COMM = 1.25*COMM
11      WHERE EMPNO = 7369;
12  END IF;
13 END;
14 /

```

PL/SQL procedure successfully completed.

```

SQL> SELECT * FROM EMP WHERE EMPNO = 7369;

```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800	3750	20

6. Write a PL/SQL block of code to print even number between 1 and 10 using for loop.

```

SQL> BEGIN
2   FOR num IN 1 .. 10 LOOP
3       IF MOD(num, 2) = 0 THEN
4           DBMS_OUTPUT.PUT_LINE(num);
5       END IF;
6   END LOOP;
7 END;
8 /

```

PL/SQL procedure successfully completed.

7. Write a PL/SQL block of code that will allow 5% salary increment of an employee (emp number should be taken from user) if the employee working in the organization more than 22 year.

```
SQL> SELECT * FROM EMP WHERE EMPNO = 7902;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

```
SQL> DECLARE
2   edate EMP.HIREDATE%TYPE;
3   eno EMP.EMPNO%TYPE;
4   BEGIN
5   eno := '&EmpNo';
6   SELECT HIREDATE INTO edate FROM EMP
7   WHERE EMPNO = eno;
8   IF SYSDATE-edate >= 22*365
9   THEN
10      UPDATE EMP SET SAL = 1.05*SAL
11      WHERE EMPNO = eno;
12      DBMS_OUTPUT.PUT_LINE('Salary Incremented Successfully.');
```

```
13   ELSE
14      DBMS_OUTPUT.PUT_LINE('Employee works for less than 22 years.');
```

```
15   END IF;
16 END;
17 /
```

```
Enter value for empno: 7902
old 5:   eno := '&EmpNo';
new 5:   eno := '7902';
Salary Incremented Successfully.
```

PL/SQL procedure successfully completed.

```
SQL> SELECT * FROM EMP WHERE EMPNO = 7902;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7902	FORD	ANALYST	7566	03-DEC-81	3150		20

Assignment-9

PL/ SQL-II: Cursors, Exception Handling:

1. Using cursor display the details of all those employees from emp table whose sum of sal and comm. Is more than 25000;

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
2  CURSOR emp_cursor IS
3  SELECT * FROM emp WHERE NVL(sal,0) + NVL(comm,0) > 25000;
4  emp_row emp%ROWTYPE;
5  BEGIN
6  OPEN emp_cursor;
7  LOOP
8  FETCH emp_cursor INTO emp_row;
9  EXIT WHEN emp_cursor%NOTFOUND;
10 DBMS_OUTPUT.PUT_LINE('Emp No: ' || emp_row.empno || ', Name: ' || emp_row.ename ||
11 ', Total Pay: ' || (NVL(emp_row.sal,0) + NVL(emp_row.comm,0)));
12 END LOOP;
13 CLOSE emp_cursor;
14 END;
15 /
Emp No: 7788, Name: SCOTT, Total Pay: 35000
PL/SQL procedure successfully completed.
```

2. Write a block which uses a Cursor For Loop to select the five highest earners from emp table and write their details to the message table which has the following three attributes: EmpNumber, Name and Salary.

```
SQL> CREATE TABLE message (
2  empnumber NUMBER,
3  name VARCHAR2(20),
4  salary NUMBER
5 );
```

Table created.

```
SQL> DECLARE
2  CURSOR c12 IS SELECT empno,sal,ename FROM emp ORDER BY sal DESC;
3  BEGIN
4  FOR rec IN c12
5  LOOP
6  INSERT INTO message (empnumber, name, salary)
7  VALUES (rec.empno, rec.ename, rec.sal);
8  EXIT WHEN c12%ROWCOUNT=5;
9  END LOOP;
10 COMMIT;
11 DBMS_OUTPUT.PUT_LINE('Top 5 earners inserted into message table.');
```

Top 5 earners inserted into message table.

PL/SQL procedure successfully completed.

```
SQL> SELECT * FROM message;
```

EMPNUMBER	NAME	SALARY
7788	SCOTT	35000
7839	KING	5000
7902	FORD	3000
7566	JONES	2975
7698	BLAKE	2850

3. Create a table Emp_sal_inc that have three column (Emp_id, Cur_sal, Inc_date). Now write a PL/SQL block of code will allow 2% salary increment of all employee of Computer Science department. After that a record is to be inserted into the above table.

```
SQL> CREATE TABLE emp_sal_inc (  
2     emp_id NUMBER,  
3     cur_sal NUMBER,  
4     inc_date DATE  
5 );
```

Table created.

```
SQL> BEGIN  
2     FOR rec IN (SELECT empno, sal FROM emp WHERE deptno IN  
3                 (SELECT deptno FROM dept WHERE dname = 'COMPUTER SCIENCE'))  
4     LOOP  
5         UPDATE emp SET sal = sal * 1.02 WHERE empno = rec.empno;  
6         INSERT INTO emp_sal_inc VALUES (rec.empno, rec.sal * 1.02, SYSDATE);  
7     END LOOP;  
8  
9     DBMS_OUTPUT.PUT_LINE('2% salary increment given and records inserted.');
```

10 END;
11 /
2% salary increment given and records inserted.

PL/SQL procedure successfully completed.

```
SQL> SELECT * FROM emp_sal_inc;
```

no rows selected

4. Write a PL/SQL block of code to update the location of specific department number that will be taken from user. Display an appropriate message using SQL%FOUND based on existence of the record in the Department table and display an appropriate message using SQL%FOUND based on the non-existence of the record in Department Table.

```

SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2   v_deptno dept.deptno%TYPE := &Enter_DeptNo;
  3   v_new_loc VARCHAR2(50) := '&Enter_New_Location';
  4 BEGIN
  5   UPDATE dept
  6   SET loc = v_new_loc
  7   WHERE deptno = v_deptno;
  8
  9   IF SQL%FOUND THEN
10     DBMS_OUTPUT.PUT_LINE('Location updated for dept no ' || v_deptno);
11   ELSE
12     DBMS_OUTPUT.PUT_LINE('No department found with dept no ' || v_deptno);
13   END IF;
14 END;
15 /
Enter value for enter_deptno: 20
old 2:   v_deptno dept.deptno%TYPE := &Enter_DeptNo;
new 2:   v_deptno dept.deptno%TYPE := 20;
Enter value for enter_new_location: 30
old 3:   v_new_loc VARCHAR2(50) := '&Enter_New_Location';
new 3:   v_new_loc VARCHAR2(50) := '30';
Location updated for dept no 20

PL/SQL procedure successfully completed.

```

```
SQL> SELECT * FROM dept;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	30
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

5. Write a PL/SQL block of code using your own exception handling that will show an error message whenever you want to insert a null value in a not null column.

```

SQL> SET SERVEROUTPUT ON;
SQL> BEGIN
  2   INSERT INTO emp (empno, ename) VALUES (NULL, 'John'); -- assuming empno is NOT NULL
  3 EXCEPTION
  4   WHEN OTHERS THEN
  5     DBMS_OUTPUT.PUT_LINE('Error: Attempted to insert NULL into a NOT NULL column.');
```

Error: Attempted to insert NULL into a NOT NULL column.

PL/SQL procedure successfully completed.

6. Write a PL/SQL block of code which raised 'NO DATA FOUND' exception when there is no employee joined in 1997 at the time of displaying 1997 joined employees.

```

SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2     v_found BOOLEAN := FALSE;
  3 BEGIN
  4     FOR rec IN (SELECT ename, hiredate FROM emp WHERE TO_CHAR(hiredate, 'YYYY') = '1997') LOOP
  5         DBMS_OUTPUT.PUT_LINE('Name: ' || rec.ename || ', Joined: ' || rec.hiredate);
  6         v_found := TRUE;
  7     END LOOP;
  8
  9     IF NOT v_found THEN
10         RAISE NO_DATA_FOUND;
11     END IF;
12 EXCEPTION
13     WHEN NO_DATA_FOUND THEN
14         DBMS_OUTPUT.PUT_LINE('No employee joined in 1997.');
```

No employee joined in 1997.

PL/SQL procedure successfully completed.

7. Write a PL/SQL Block of code to find the QOH amount from item_master table. If QOH is less than 1 raise an user define exception “INSUFFICIENT STOCK”.

Assignment 10: Procedures, Functions and Triggers:

1. Create a Procedure INCR_OF_EMP in which you will pass empno and amount of increment. If the salary of the specified employee is NULL in Employee table raised the exception 'Salary Missing' and display the message 'This Employee has NULL salary' else update the salary of the employee. Invoke the INCR_OF_EMP procedure from SQL prompt with some empno and increment amount and follow the consequences.

```
CREATE OR REPLACE PROCEDURE INCR_OF_EMP (
    p_empno IN EMP.EMPNO%TYPE,
    p_increment IN NUMBER
)
IS
    v_salary EMP.SAL%TYPE;
BEGIN
    SELECT SAL INTO v_salary FROM EMP WHERE EMPNO = p_empno;

    IF v_salary IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001, 'This Employee has NULL salary');
    ELSE
        UPDATE EMP SET SAL = SAL + p_increment WHERE EMPNO = p_empno;
        DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');

```

Procedure created.

```
BEGIN
    INCR_OF_EMP(7369, 1000);
END;
```

Statement processed.
Salary updated successfully.

2. Create a function REVIEW_EMP in which we will pass empno as parameter and return the incremented values of salary based on the following condition: If salary <= 15000 then increment will be 20%, If salary > 15000 and salary <= 30000 then increment will be 30% Else increment will be 40%. Call the REVIEW_EMP function from a PL/SQL Block with empno = 7698 and display the result on the screen.

```
CREATE OR REPLACE FUNCTION REVIEW_EMP (
    p_empno IN EMP.EMPNO%TYPE
) RETURN NUMBER
IS
    v_salary EMP.SAL%TYPE;
    v_new_salary NUMBER;
BEGIN
    SELECT SAL INTO v_salary FROM EMP WHERE EMPNO = p_empno;

    IF v_salary <= 15000 THEN
        v_new_salary := v_salary * 1.2;
    ELSEIF v_salary <= 30000 THEN
        v_new_salary := v_salary * 1.3;
    ELSE
        v_new_salary := v_salary * 1.4;
    END IF;

    RETURN v_new_salary;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');

```

Function created.

```
DECLARE
    v_result NUMBER;
BEGIN
    v_result := REVIEW_EMP(7698);
    DBMS_OUTPUT.PUT_LINE('New Salary: ' || v_result);
END;
```

Statement processed.
New Salary: 3428

3. Create a trigger UPPER_STRING for the employee table, which makes the entry in ENAME column in UPPER CASE.

```
CREATE OR REPLACE TRIGGER UPPER_STRING
BEFORE INSERT OR UPDATE OF ENAME ON EMP
FOR EACH ROW
BEGIN
    :NEW.ENAME := UPPER(:NEW.ENAME);
END;
```

Trigger created.

4. Write a trigger TOTAL_SALARY to maintain a derived column TOTSAL of Dept table that stores total salary of all employees in a department.

```
CREATE OR REPLACE TRIGGER TOTAL_SALARY
AFTER INSERT OR DELETE OR UPDATE OF SAL ON EMP
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        UPDATE DEPT SET TOTSAL = NVL(TOTSAL, 0) + :NEW.SAL WHERE DEPTNO = :NEW.DEPTNO;
    ELSEIF DELETING THEN
        UPDATE DEPT SET TOTSAL = NVL(TOTSAL, 0) - :OLD.SAL WHERE DEPTNO = :OLD.DEPTNO;
    ELSEIF UPDATING THEN
        IF :OLD.DEPTNO = :NEW.DEPTNO THEN
            UPDATE DEPT SET TOTSAL = TOTSAL - :OLD.SAL + :NEW.SAL WHERE DEPTNO = :NEW.DEPTNO;
        ELSE
            UPDATE DEPT SET TOTSAL = TOTSAL - :OLD.SAL WHERE DEPTNO = :OLD.DEPTNO;
            UPDATE DEPT SET TOTSAL = NVL(TOTSAL, 0) + :NEW.SAL WHERE DEPTNO = :NEW.DEPTNO;
        END IF;
    END IF;
END;
```

Trigger created.

5. Create a INSTEAD OF TRIGGER in which, if a delete statement is fired on a VIEW, say V1 for an employee Name, then the corresponding employee's records gets deleted from Employee table.

6. Write a SQL query by which you will see the code that defines the TOTAL_SALARY trigger.

```
CREATE OR REPLACE VIEW VI AS
SELECT EMPNO, ENAME, SAL, DEPTNO FROM EMP
```

View created.

```
CREATE OR REPLACE TRIGGER INSTEAD_OF_DELETE_V1
INSTEAD OF DELETE ON VI
FOR EACH ROW
BEGIN
    DELETE FROM EMP WHERE ENAME = :OLD.ENAME;
END;
```

Trigger created.

```
SELECT TEXT
FROM USER_SOURCE
WHERE NAME = 'TOTAL_SALARY'
AND TYPE = 'TRIGGER'
```

```
TEXT

TRIGGER TOTAL_SALARY

AFTER INSERT OR DELETE OR UPDATE OF SAL ON EMP

FOR EACH ROW

BEGIN

IF INSERTING THEN

UPDATE DEPT SET TOTALSAL = NVL(TOTALSAL, 0) + (NEW.SAL WHERE DEPTNO = (NEW.DEPTNO));

ELSIF DELETING THEN

UPDATE DEPT SET TOTALSAL = NVL(TOTALSAL, 0) - (OLD.SAL WHERE DEPTNO = (OLD.DEPTNO));

ELSIF UPDATING THEN

IF ((OLD.DEPTNO <> NEW.DEPTNO) THEN

UPDATE DEPT SET TOTALSAL = TOTALSAL - (OLD.SAL + (NEW.SAL WHERE DEPTNO = (NEW.DEPTNO));

ELSE

UPDATE DEPT SET TOTALSAL = TOTALSAL - (OLD.SAL WHERE DEPTNO = (OLD.DEPTNO));

UPDATE DEPT SET TOTALSAL = NVL(TOTALSAL, 0) + (NEW.SAL WHERE DEPTNO = (NEW.DEPTNO));

END IF;

END IF;
```

7. Drop the TOTAL_SALARY trigger from the database.

```
DROP TRIGGER TOTAL_SALARY
```

Trigger dropped.

Assignment-11 : User creation with privilege options:

1. Write a SQL statement to create a new local user named 'Test1_<roll_no>' e.g Test1_007 with the password 'mckvie2024' and with password expired option.

```
CREATE USER Test1_100 IDENTIFIED BY mckvie2024 PASSWORD EXPIRE;
```

But for Oracle express edition

```
CREATE USER CR#Test1_100 IDENTIFIED BY mckvie2024 PASSWORD EXPIRE;
```

OR

```
CREATE USER C##Test1_100 IDENTIFIED BY mckvie2024 PASSWORD EXPIRE;
```

2. Find a list of users (username, default_tablespace, profile, authentication_type) with the OPEN status; you query the information from the dba_users.

```
SELECT
  username,
  default_tablespace,
  profile,
  authentication_type
FROM
  dba_users
WHERE
  account_status = 'OPEN';
```

3. Try to login to the newly created user using a new window and observe the outcome and comment on it.

4. Write a SQL statement to enable with the login system privilege to the user 'Test1_<roll_no>'

```
GRANT CREATE SESSION TO Test1_100;
```

```
CONNECT Test1_100
PASSWORD: mckvie2024
ERROR:
```

Enter user-name: Test1_100 Password: mckvie2024

```
Oracle issued the following error:
ERROR: ORA-01045:
user Test1_100 lacks CREATE SESSION privilege;
login denied
```

ORA-28001: the password has expired

Oracle requested to change the password for Test1_100, we must provide the new password and confirm it before we can log in:

```
Changing password for Test1_100
New password:<new_password>
Retype new password:<new_password>
Password changed
Connected.
SQL>
```

5. Write a SQL statement to grant the CREATE TABLE system privilege to the user 'Test1_<roll_no>'.

```
GRANT CREATE TABLE TO Test1_100;
```

6. Write a SQL statement to statement to show the privileges of the current user.

```
SELECT * FROM session_privs;
```

7. Create a new table named t2 (ID NUMBER (4)) and allow 'Test1_<roll_no>' to insert and update data in it.

```
CREATE TABLE t2(ID NUMBER (4));

INSERT INTO t2(ID) VALUES(1111);

Oracle issued the following error:
ORA-01950: no privileges on tablespace 'USERS'

This is because the user john has a quota of zero on the USERS tablespace.

ALTER USER Test1_100 QUOTA UNLIMITED ON USERS;

Now, the user Test1_100 should be able to insert a row into the t2 table:

INSERT INTO t2(ID) VALUES(1111);
```

8. Write a SQL statement to revoke the object privileges from 'Test1_<roll_no>'.

```
REVOKE ALL PRIVILEGES FROM Test1_100;
```